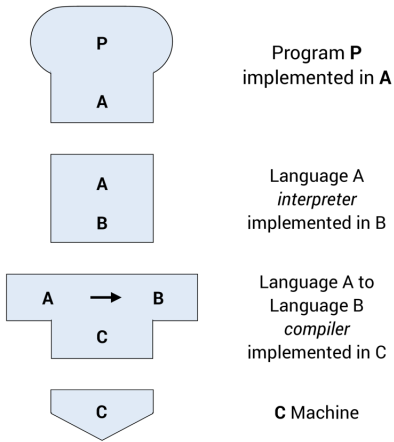


Programming Languages

Dynamic vs Static Typing: Dynamic – variables can hold values of multiple different unrelated types. Static – variable types must be declared and cannot be changed.

Strong vs Weak Typing: Stronger languages have greater rules in their type system to ensure type safety.

Tombstone Diagram



Subtyping

T is a subtype of S ($T \leq S$) if code written for S can be safely used for T. T has narrower scope, S has wider scope.

- **Widening:** T can be put into S
- **Narrowing:** S can be explicitly typecast into T
- **Reflexive:** $T \leq T$
- **Transitive:** $T \leq S, S \leq U \Rightarrow T \leq U$
- **Anti-Symmetric:** $T \leq S, S \leq T \Rightarrow S = T$

Primitive Types

- **Numeric:** byte $\leq$ short $\leq$ int $\leq$ long $\leq$ float $\leq$ double
- **Character:** char $\leq$ int
- **Boolean:** boolean
- **Void:** void (no value)

Reference Types

Anything not a primitive type (e.g., classes). Stores only a reference to the value. Two reference variables can share the same value – equality by default compares references. Uninitialized: null value, throws NullPointerException.

01 OOP Fundamentals

Encapsulation

Bundle data and methods within a single unit (class). Hide internal state.

```
class Circle {  
    private double radius;  
  
    public Circle(double radius) {  
        this.radius = radius;  
    }  
}
```

```
public double getRadius() { return radius; }  
public double getArea() {  
    return Math.PI * radius * radius;  
}
```

Immutability

Object state cannot be modified after creation. Use final.

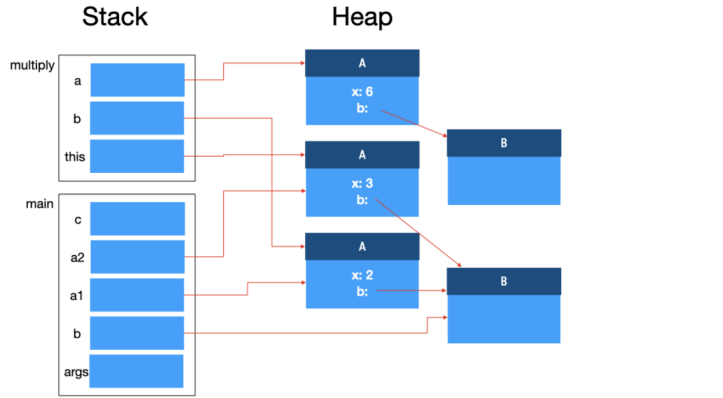
Tell, Don't Ask

Tell objects what to do; don't ask for their state.

```
// Bad  
if (circle.getRadius() > 5) { /* ... */ }  
  
// Good  
circle.isLargerThan(5.0)
```

Stack and Heap

Stack: Where all variables are allocated and stored. Contains **Call Frames**, created when methods are invoked and destroyed when returned.
Heap: Where objects are allocated and stored. Whenever new is used, a new object is created on the heap. Objects store: Class Name, Instance Fields and Values, Captured Variables.



Class Fields and Methods

- Use static keyword for class-level fields/methods
- Fields accessed via `Class.field` (shared across entire class)
- Methods accessed via `Class.method()`, but this will **NOT** work

02 Inheritance & Polymorphism

Inheritance

```
class Circle extends Shape {  
    private double radius;  
  
    public Circle(double radius, String color) {  
        super(color); // MUST BE FIRST LINE  
        this.radius = radius;  
    }  
  
    @Override  
    double getArea() {  
        return Math.PI * radius * radius;  
    }  
}
```

Feature	Overloading (Static)	Overriding (Dynamic)
Scope	Same class or inherited	Between subclass & superclass
Signature	Different params (type, order, #)	Same params & name
Return Type	Can be different	Covariant ($Ret_{sub} \leq Ret_{parent}$)
Access	Can be different	Non-weaker (same or more open)
Exceptions	Can be different	Fewer or more specific checked
Binding	Compile-Time (CTT of args)	Run-Time (RTT of target)

Polymorphism

Same interface, different implementations.

```
Shape s1 = new Circle(5.0, "red");  
Shape s2 = new Rectangle(3.0, 4.0, "blue");  
  
double area1 = s1.getArea(); // Circle's getArea  
double area2 = s2.getArea(); // Rectangle's getArea
```

Abstract Classes

A general class that should **NEVER** be instantiated. Abstract arrays can still be defined. Abstract methods have no body. A class with abstract methods must be made abstract.

```
abstract class Shape {  
    abstract double getArea();  
    abstract double getPerimeter();  
  
    // Concrete method  
    public void printInfo() {  
        System.out.println("Area: " + getArea());  
    }  
}
```

Interfaces

Models requirements of classes. Methods are public abstract by default. A class can implement multiple interfaces (solves Diamond Problem). Can be impure using default keyword.

```
interface Drawable {  
    void draw();  
    default void info() { /* body */ } // impure  
}  
  
class Circle implements Drawable, Comparable<Circle> {  
    public void draw() { /* ... */ }  
    public int compareTo(Circle other) {  
        return Double.compare(this.radius, other.radius);  
    }  
}
```

Liskov Substitution Principle (LSP)

Any property of objects of type T should be true for any object of type S where $S \leq T$. Don't break expectations of the parent class. Can use final to prevent inheritance or modification of methods/fields.

```
void processShape(Shape s) {  
    double area = s.getArea(); // works for all  
}  
  
processShape(new Circle(5.0));  
processShape(new Rectangle(3.0, 4.0));
```

Dynamic Binding

CTT (Compile Time Type): Declared type of variable.

RTT (Run Time Type): Actual type of object at runtime.

```
Circle c = new ColouredCircle();
// CTT(c) = Circle, RTT(c) = ColouredCircle
```

For obj.foo(arg):

Compile Time:

- Check CTT(obj); find all accessible foo methods (including supertypes of CTT(obj))
- Find most specific method fitting CTT(arg); record method descriptor

Run Time:

- Retrieve method descriptor; determine RTT(obj)
- From RTT(obj), find first fitting method descriptor; recurse upwards until found

Does **not** apply to class (static) methods.

Wrapper Classes

Java provides wrapper classes for primitive types (e.g., Integer, Double).

```
Integer i = 4; // auto-boxing (line 1)
int j = i;     // auto-unboxing (line 2)
```

- **Auto-boxing:** Automatically put primitive into wrapper class
- **Auto-unboxing:** Automatically get value from wrapper class
- 2-step boxing **NOT** allowed (e.g., Double d = 2)
- Wrapper classes **DO NOT** share subtyping with primitives
- Performance cost: objects stored on heap; wrapper classes are immutable

Type Casting & Checking

a = (C) b; where b has CTT(b) and RTT(b).

Compile-Time Check (The “Is it Possible?” Rule)

The compiler allows the cast if there is any theoretical chance RTT(b) <: C.

- **Widening:** CTT(b) <: C. Always allowed; no runtime check needed.
- **Narrowing:** C <: CTT(b). Allowed; requires runtime check (ClassCastException if fails).
- **Interface Exception:** If C is an interface and CTT(b) is a non-final class, the cast is *always* allowed at compile-time. Reason: A future subclass of CTT(b) could implement C.
- **Cross-Casting:** Casting between two unrelated interfaces is always allowed at compile-time for the same reason (a class could implement both).
- **Hard Fail:** If C and CTT(b) are unrelated classes, OR CTT(b) is final and does not implement interface C.

Run-Time Check

The JVM ignores CTT and looks only at the heap:

- Check: RTT(b) <: C?
- **Success:** Execution continues; CTT(a) is now C.
- **Failure:** Throws ClassCastException.

Variance

- **Covariant:** S <: T implies S[] <: T[] (Java arrays are covariant)
- **Contravariant:** S <: T implies (T) <: (S)
- **Invariant:** None of the above (Java generics are invariant)

03 Generics

Generic Class

```
class Box<T> {
    private T item;

    public Box(T item) { this.item = item; }
    public T getItem() { return item; }

    public <U> Box<U> map(Transformer<T, U> f) {
```

```
        return new Box<>(f.transform(item));
    }
}
```

```
Box<String> strBox = new Box<>("hello");
```

Generic Methods

```
public static <T extends Comparable<T>> T max(T a, T b) {
    return a.compareTo(b) > 0 ? a : b;
}
```

Bounded Type Parameters

```
// Upper bound: T must be Number or subclass
class NumBox<T extends Number> {
    private T value;
    public double getDouble() {
        return value.doubleValue();
    }
}
// Multiple bounds
<T extends Shape & Drawable>
```

Type Erasure

After type checking, Java performs type erasure for backwards compatibility. Generic types replaced by upper bound (default Object). When a Generic is instantiated, a typecast is added.

```
// Before erasure
Int i = new Pair<Str,Int>("a", 4).sec();
// After erasure
Int i = (Int) new Pair("a", 4).sec();
```

Cannot use: new T(), new T[], T.class

Generic Array Problems

Due to type erasure, cannot tell if putting Generics into an array will cause errors. Generic array declaration is ok, but instantiation with new is not.

```
// Problem: becomes Pr[] after erasure
Pr<Str,Int>[] pArr = new Pr<Str,Int>[2];
Object[] oArr = pArr;
oArr[0] = new Pr<Db1,Bool>(3.14, true);
// -> ClassCastException
```

Arrays are reifiable (full type info at runtime); generics are not, so Java does not allow creating generic arrays with new.

Generic Arrays and Warnings

Since we cannot instantiate generic arrays, typecast instead (arrays are covariant):

```
T[] tmp = (T[]) new Object[sz];
this.arr = tmp;
```

Generates unchecked warning. Suppress with

@SuppressWarnings("unchecked") only after justifying: array is private so only accessed by the class; array can only be modified by Seq::set(int, T), which is of type T; the only retrievable objects from array are subtypes of T via Seq::get(int).

Generic Type Rules

- Generic method signature includes type parameters (equal up to renaming)
- Type checking uses type argument for class-level type parameters where possible
- Method descriptor stored during dynamic binding at CTT is type erased

Wildcards

```
// Upper bounded (covariant)
// copyFrom(Seq<? extends T> src): src is subtype of T
void processShapes(List<? extends Shape> shapes) {
    for (Shape s : shapes) { s.getArea(); } // can read
    // shapes.add(new Circle()); // CANNOT add
}
```

```
// Lower bounded (contravariant)
// copyTo(Seq<? super T> dest): dest is supertype of T
void addCircles(List<? super Circle> list) {
    list.add(new Circle(5.0)); // can add
    // Circle c = list.get(0); // CANNOT get
}
```

PECS: Producer Extends, Consumer Super

? extends T for **reading** (producer); ? super T for **writing** (consumer)

Unbounded wildcards: Similar to Object supertype, but object as a generic type parameter does not work as generics are invariant. Used to replace raw types.

Type Inference

Use empty diamond <> on RHS to instantiate generic types. 3-step method:

1. **Argument Typing:** Type of parameters passed in
2. **Target Typing:** Type of variable where return is stored
3. **Type Parameter:** Explicitly specified type parameters in the declaration

Java considers types explicitly involved in inequalities, then chooses the most specific type.

Bridge Methods

Compiler generates bridge methods to preserve polymorphism after type erasure when inheritance and overriding interact with generics.

Conditions: (i) Class is subtype of parameterised type; (ii) Type erasure changes inherited method signature.

```
class Box<T> { void get(T t) {} }
class StringBox extends Box<String> {
    @Override void get(String s) {} // user code
}
// After erasure, compiler generates:
class StringBox extends Box {
    void get(Object s) { this.get((String) s); } // bridge
    void get(String s) {} // user code
}
```

06 Exception Handling

Try-Catch-Finally

```
try {
    int result = riskyOperation();
} catch (SpecificException e) {
    // Handle specific
} catch (Exception e) {
    // Handle general
} finally {
    // Always executed (cleanup)
}
```

Checked: Must be caught/declared (Exception)

Unchecked: Runtime exceptions (RuntimeException)

throw immediately suspends try. finally **ALWAYS RUNS**.

Try-with-Resources

```
try (FileReader fr = new FileReader("f.txt")) {
    String line = br.readLine();
} // Resources auto-closed
```